

Lab 7: Deploying to Kubernetes with Terraform

In this lab, you will create a Terraform module that can be used to deploy a website that is packaged in a Docker container. You will then use this module to deploy a site to a Kubernetes cluster running in AWS.

Step 1: Creating a Kubernetes Deployment Module

1. In your `~/terraform/aws/` folder, create a directory named `kubernetes`. Inside it, create a folder named `modules`, and inside it, one named `website-config`. Then, switch to the `~/terraform/aws/kubernetes/modules/website-config` folder. You can use the following code if you like.

```
mkdir ~/terraform/aws/kubernetes
mkdir ~/terraform/aws/kubernetes/modules
mkdir ~/terraform/aws/kubernetes/modules/website-config
cd ~/terraform/aws/kubernetes/modules/website-config
```

2. Create the following files: `kubernetes-deployment.tf`, `kubernetes-load-balancer.tf`, `kubernetes-namespace.tf`, `output.tf`, and `variables.tf`. You can use the following command.

```
touch kubernetes-deployment.tf kubernetes-load-balancer.tf
kubernetes-namespace.tf output.tf variables.tf
```

3. Enter `ls` to verify the files were created.



4. Add the following to the `kubernetes-deployment.tf` file. Be sure to replace your initials for `<Your initials>` in the deployment name and all subsequent objects you create in this lab.

```
resource "kubernetes_deployment" "deployment" {
  metadata {
    name = "<Your initials>-${var.project}-deployment"
    labels = {
      App = var.project
    }
    namespace = kubernetes_namespace.n.metadata[0].name
  }

  spec {
    replicas = 3
    progress_deadline_seconds = 300
    selector {
      match_labels = {
        App = var.project
      }
    }
  }

  template {
    metadata {
      labels = {
        App = var.project
      }
    }
    spec {
      container {
        image = var.container
        name = var.project

        port {
          container_port = 80
        }

        resources {
          limits = {
```

```
        cpu    = "0.2"
        memory = "256Mi"
      }
      requests = {
        cpu    = "0.1"
        memory = "50Mi"
      }
    }
  }
}
}
```

5. Add the following to the `kubernetes-load-balancer.tf` file.

```
resource "kubernetes_service" "service" {
  metadata {
    name      = "<Your initials>-${var.project}-service"
    namespace = kubernetes_namespace.n.metadata[0].name
  }
  spec {
    selector = {
      App =
kubernetes_deployment.deployment.spec.0.template.0.metadata[0].labels
.App
    }
    port {
      port          = 80
      target_port = 80
    }

    type = "LoadBalancer"
  }
}
```

6. Add the following to the `kubernetes-namespace.tf` file.

```
resource "kubernetes_namespace" "n" {  
  metadata {  
    name = "<Your initials>-${var.project}-ns"  
  }  
}
```

7. Add the following to the `output.tf` file.

```
output "lb_status" {  
  value = kubernetes_service.service.status  
}
```

8. Add the following variables to the `variables.tf` file.

```
variable "project" {  
  type      = string  
  description = "Application Name"  
}  
  
variable "container" {  
  type      = string  
  description = "The Docker Container to Deploy."  
}
```

Note: This module is complete. It can be used to deploy a website using Kubernetes. The variables that need to be supplied are simply a name for the website and a Docker Image that contains it.

Note: The completed code for this module is located here:

<https://github.com/roitraining/hands-on-terraform/tree/main/kubernetes/module/s/website-config>

Step 2: Deploying to an EKS Cluster with Terraform

1. In the `kubernetes` folder, create a folder named `eks` and change to it.

```
mkdir ~/terraform/aws/kubernetes/eks
cd ~/terraform/aws/kubernetes/eks
```

2. Create the following files: `cluster.tf`, `main.tf`, `output.tf`, `providers.tf`, `terraform.tfvars`, and `variables.tf`. You can use the following command.

```
touch cluster.tf main.tf output.tf providers.tf terraform.tfvars
variables.tf
```

3. Enter `ls` to verify the files were created.
4. Add the following variable to the `variables.tf` file. Be sure to change the region to the classroom region.

```
variable "region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}
```

5. Open the `providers.tf` file. This time, you need to configure two providers: the AWS one as usual and the Terraform provider. Since you are using AWS in this example, the Terraform provider will be configured to read from your EKS cluster. Add the following to the file.

```
terraform {  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.47.0"  
    }  
  
    random = {  
      source  = "hashicorp/random"  
      version = "~> 3.6.1"  
    }  
  
    tls = {  
      source  = "hashicorp/tls"  
      version = "~> 4.0.5"  
    }  
  
    cloudinit = {  
      source  = "hashicorp/cloudinit"  
      version = "~> 2.3.4"  
    }  
  
    kubernetes = {  
      source  = "hashicorp/kubernetes"  
      version = "2.38.0"  
    }  
  }  
  
  required_version = "~> 1.3"  
}  
  
provider "aws" {  
  region = var.region  
}  
  
provider "kubernetes" {
```

```
host          = module.eks.cluster_endpoint
cluster_ca_certificate =
base64decode(module.eks.cluster_certificate_authority_data)
exec {
  api_version = "client.authentication.k8s.io/v1beta1"
  command     = "aws"
  args = [
    "eks", "get-token",
    "--cluster-name", module.eks.cluster_name,
    "--region", var.region
  ]
}
```

6. In `cluster.tf`, add the following configuration to create the EKS cluster. Be sure to replace `<Your initials>` everywhere in the file as you have been doing with your actual initials.

```
# Filter out local zones, which are not currently supported
# with managed node groups
data "aws_availability_zones" "available" {
  filter {
    name     = "opt-in-status"
    values   = ["opt-in-not-required"]
  }
}

locals {
  cluster_name = "<Your
initials>-eks-cluster-${random_string.suffix.result}"
}

resource "random_string" "suffix" {
  length  = 8
  special = false
}
```

```
}

module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"

  name = "<Your iinitials>-education-vpc"

  cidr = "10.0.0.0/16"
  azs   = slice(data.aws_availability_zones.available.names, 0, 3)

  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.4.0/24", "10.0.5.0/24", "10.0.6.0/24"]

  enable_nat_gateway  = true
  single_nat_gateway  = true
  enable_dns_hostnames = true

  public_subnet_tags = {
    "kubernetes.io/role/elb" = 1
  }

  private_subnet_tags = {
    "kubernetes.io/role/internal-elb" = 1
  }
}

module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "20.8.5"

  cluster_name      = local.cluster_name
  cluster_version    = "1.34"

  cluster_endpoint_public_access = true
  enable_cluster_creator_admin_permissions = true

  cluster_addons = {
    aws-ebs-csi-driver = {
```



```
        service_account_role_arn = module.irsaebscsi.iam_role_arn
    }
}

vpc_id      = module.vpc.vpc_id
subnet_ids  = module.vpc.private_subnets

eks_managed_node_group_defaults = {
    ami_type = "AL2023_x86_64_STANDARD"
}

eks_managed_node_groups = {
    one = {
        name = "<Your initials>-node-group-1"

        instance_types = ["t3.small"]

        min_size      = 1
        max_size       = 3
        desired_size    = 2
    }

    two = {
        name = "<Your initials>-node-group-2"

        instance_types = ["t3.small"]

        min_size      = 1
        max_size       = 2
        desired_size    = 1
    }
}

#
https://aws.amazon.com/blogs/containers/amazon-ebs-csi-driver-is-now-generally-available-in-amazon-eks-add-ons/
```

```
data "aws_iam_policy" "ebs_csi_policy" {
  arn =
  "arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy"
}

module "irsa-ebs-csi" {
  source =
  "terraform-aws-modules/iam/aws//modules/iam-assumable-role-with-oidc"
  version = "5.39.0"

  create_role          = true
  role_name            =
  "AmazonEKSTFEBSCSIRole-${module.eks.cluster_name}"
  provider_url         = module.eks.oidc_provider
  role_policy_arns     =
  [data.aws_iam_policy.ebs_csi_policy.arn]
  oidc_fully_qualified_subjects =
  ["system:serviceaccount:kube-system:ebs-csi-controller-sa"]
}
```

7. You need to import your module and deploy your website. Open `main.tf` and add the following configuration. Note the container variable. This is the Docker image that has the website inside it.

```
module "kubernetes_app" {
  source      = "../modules/website-config" # <- your module path
  project     = "space-invaders"
  container   = "drehnstrom/space-invaders:v1.0"

  # wait until cluster + aws-auth exist
  depends_on = [module.eks]
}
```

8. Finally, add the following configuration to the `output.tf` file. This will return the IP address of the website's load balancer.

```
output "cluster_endpoint" {
  description = "Endpoint for EKS control plane"
  value      = module.eks.cluster_endpoint
}

output "cluster_security_group_id" {
  description = "Security group ids attached to the cluster control plane"
  value      = module.eks.cluster_security_group_id
}

output "region" {
  description = "AWS region"
  value      = var.region
}

output "cluster_name" {
  description = "Kubernetes Cluster Name"
  value      = module.eks.cluster_name
}
```

9. Run the Terraform `init` and `plan` commands. If there are no errors, `apply` your configuration. Make sure you are in the right folder when running these commands.
10. Once everything is deployed, update the kubeconfig file for use with the `kubectl` commands in the next few steps by running this command.

```
aws eks --region $(terraform output -raw region) update-kubeconfig
--name $(terraform output -raw cluster_name)
```

11. View the list of namespaces in Kubernetes by running this command.

```
kubectl get ns
```

12. Make a note of the namespace that has `space-invaders-ns` in it for the next two commands and use that in place of the place holder `<namespace>` in those commands.

13. Run the following command to get the list of pods running in the game namespace.

```
kubect1 get pods -n <namespace>
```

14. Finally, you will need the load balancer DNS name to open the game and verify that everything worked properly. You can obtain that DNS name with the following command.

```
kubect1 get services -n <namespace>
```

15. Copy the load balancer name and paste it into a browser to test it. It may take a minute for the pods to be loaded and initialized before the game is ready to be played.

16. Run the `Terraform destroy` command to delete what you created.

Note: The completed code for this lab is located here:

<https://github.com/roitraining/hands-on-terraform/tree/main/kubernetes/eks>